

LAB EXPERIMENTS 31-40

Q.31:

PROGRAM:

```
#include <stdio.h>
#include <stdint.h>

uint64_t leftShift64(uint64_t x) {
    return x << 1;
}

uint64_t generateK2(uint64_t K1, uint64_t Rb) {
    uint64_t K2 = K1 << 1;

    if (K1 & 0x8000000000000000ULL)
        K2 ^= Rb;

    return K2;
}

int main() {
    uint64_t L, K1, K2;

    printf("Enter 64-bit L in hexadecimal: ");
    scanf("%llx", &L);

    K1 = L << 1;

    if (L & 0x8000000000000000ULL)
        K1 ^= 0x1B;

    K2 = generateK2(K1, 0x1B);

    printf("K1 = %016llx\n", K1);
    printf("K2 = %016llx\n", K2);
}
```

OUTPUT:

```
Enter 64-bit L in hexadecimal: 800000000000000000000000
K1 = 000000000000000000000000
K2 = 000000000000000000000000

CMAC constants:
64-bit = 0x1B
128-bit = 0x87

-----
Process exited after 38.91 seconds with return value 0
Press any key to continue . . . |
```

Q.32:

PROGRAM:

```
#include <stdio.h>

int main() {
    int r1, s1, r2, s2;

    printf("Enter first DSA signature (r s): ");
    scanf("%d %d", &r1, &s1);

    printf("Enter second DSA signature (r s): ");
    scanf("%d %d", &r2, &s2);

    printf("\nSignature 1 = (%d, %d)\n", r1, s1);
    printf("Signature 2 = (%d, %d)\n", r2, s2);

    if (r1 != r2 || s1 != s2)
        printf("Signatures are different.\n");
    else
        printf("Signatures are identical.\n");

    printf("\nReason: DSA normally uses a fresh random k for each signature.\n");
    printf("RSA signatures with deterministic signing can be identical\n");
    printf("for the same message and key.\n");

    return 0;
}
```

OUTPUT:

```
Enter first DSA signature (r s): 12 45
Enter second DSA signature (r s): 59 56

Signature 1 = (12, 45)
Signature 2 = (59, 56)
Signatures are different.

Reason: DSA normally uses a fresh random k for each signature.
RSA signatures with deterministic signing can be identical
for the same message and key.

-----
Process exited after 44.85 seconds with return value 0
Press any key to continue . . . |
```

Q.33:

PROGRAM:

```
int main() {
    uint64_t plaintext, key, cipher, decrypted;

    printf("Enter 64-bit plaintext in hexadecimal: ");
    scanf("%llx", &plaintext);

    printf("Enter 64-bit key in hexadecimal: ");
    scanf("%llx", &key);

    cipher = des_demo(plaintext, key);

    printf("Ciphertext = %016llx\n", cipher);

    /*
     * For this demonstration transform, applying
     * the inverse rotation/XOR operation would be
     * used for decryption.
     */

    decrypted = cipher;

    for (int i = 15; i >= 0; i--) {
        decrypted = (decrypted >> 1) |
                   (decrypted << 63);
        decrypted ^= (key << (i % 8));
    }

    printf("Decrypted = %016llx\n", decrypted);

    return 0;
}
```

```
Enter 64-bit plaintext in hexadecimal: 123456AGBECS
Enter 64-bit key in hexadecimal: Ciphertext = 00000123456A0000
Decrypted = 0000000000123456A
```

```
-----
Process exited after 37.8 seconds with return value 0
Press any key to continue . . . |
```

Q.35:

PROGRAM:

```
#include <stdio.h>
#include <string.h>

int main() {
    char plaintext[100], ciphertext[100], decrypted[100];
    int key[100], n;

    printf("Enter plaintext: ");
    fgets(plaintext, sizeof(plaintext), stdin);

    plaintext[strcspn(plaintext, "\n")] = '\0';

    n = strlen(plaintext);

    printf("Enter %d key values (0-25): ", n);

    for (int i = 0; i < n; i++)
        scanf("%d", &key[i]);

    for (int i = 0; i < n; i++) {
        if (plaintext[i] >= 'A' && plaintext[i] <= 'Z')
            ciphertext[i] =
                ((plaintext[i] - 'A' + key[i]) % 26) + 'A';
        else
            ciphertext[i] = plaintext[i];
    }

    ciphertext[n] = '\0';

    printf("Ciphertext: %s\n", ciphertext);

    for (int i = 0; i < n; i++) {
```

OUTPUT:

```
Enter plaintext: ABC
Enter 3 key values (0-25): 3 15 20
Ciphertext: DQW
Decrypted : ABC

-----
Process exited after 21.44 seconds with return value 0
Press any key to continue . . . |
```

Q.36:

PROGRAM:

```
        return -1;
    }

int main() {
    char text[100];
    int a, b, inv;

    printf("Enter plaintext: ");
    fgets(text, sizeof(text), stdin);

    printf("Enter a: ");
    scanf("%d", &a);

    printf("Enter b: ");
    scanf("%d", &b);

    if (gcd(a, 26) != 1) {
        printf("Invalid a. gcd(a,26) must be 1.\n");
        return 0;
    }

    inv = inverse(a);

    for (int i = 0; text[i]; i++) {
        if (text[i] >= 'A' && text[i] <= 'Z')
            text[i] = ((a * (text[i] - 'A') + b) % 26) + 'A';
    }

    printf("Ciphertext: %s", text);

    return 0;
}
```

OUTPUT:

```
Enter plaintext: HELLO
Enter a: 5
Enter b: 8
Ciphertext: RCLLA

-----
Process exited after 11.73 seconds with return value 0
Press any key to continue . . . |
```

Q.37:

PROGRAM:

```
#include <ctype.h>

int main() {
    char text[1000];
    int freq[26] = {0};
    int total = 0;

    printf("Enter ciphertext:\n");
    fgets(text, sizeof(text), stdin);

    for (int i = 0; text[i]; i++) {
        if (isalpha((unsigned char)text[i])) {
            char c = toupper((unsigned char)text[i]);
            freq[c - 'A']++;
            total++;
        }
    }

    printf("\nLetter frequencies:\n");

    for (int i = 0; i < 26; i++) {
        if (freq[i] > 0)
            printf("%c : %.2f%%\n",
                    'A' + i,
                    100.0 * freq[i] / total);
    }

    printf("\nMost frequent letters are possible substitutes\n");
    printf("for common English letters such as E, T, A and O.\n");

    return 0;
}
```

OUTPUT:

```
Enter ciphertext:
WEXDF TSXF TRE

Letter frequencies:
D : 8.33%
E : 16.67%
F : 16.67%
R : 8.33%
S : 8.33%
T : 16.67%
W : 8.33%
X : 16.67%

Most frequent letters are possible substitutes
for common English letters such as E, T, A and O.

-----
Process exited after 22.65 seconds with return value 0
Press any key to continue . . . |
```

Q.38:

PROGRAM:

```
det = mod26(p[0][0] * p[1][1] -
           p[0][1] * p[1][0]);

invDet = inverseMod(det);

if (invDet == -1) {
    printf("Plaintext matrix is not invertible.\n");
    return 0;
}

inv[0][0] = mod26(p[1][1] * invDet);
inv[0][1] = mod26(-p[0][1] * invDet);
inv[1][0] = mod26(-p[1][0] * invDet);
inv[1][1] = mod26(p[0][0] * invDet);

for (int i = 0; i < 2; i++)
    for (int j = 0; j < 2; j++)
        k[i][j] = mod26(
            c[i][0] * inv[0][j] +
            c[i][1] * inv[1][j]);

printf("Recovered Key Matrix:\n");

for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 2; j++)
        printf("%d ", k[i][j]);
    printf("\n");
}

return 0;
```

OUTPUT:

```
Enter plaintext matrix:
1 2
3 5
Enter ciphertext matrix:
3 6
7 8
Recovered Key Matrix:
3 0
15 6

-----
Process exited after 28.73 seconds with return value 0
Press any key to continue . . . |
```

Q.39:

PROGRAM:

```
#include <stdio.h>
#include <string.h>

void decrypt(char *cipher, int shift) {
    for (int i = 0; cipher[i]; i++) {
        if (cipher[i] >= 'A' && cipher[i] <= 'Z')
            putchar((cipher[i] - 'A' - shift + 26) % 26 + 'A');
        else
            putchar(cipher[i]);
    }
}

int main() {
    char cipher[500];

    printf("Enter ciphertext: ");
    fgets(cipher, sizeof(cipher), stdin);

    printf("\nPossible plaintexts:\n");

    for (int shift = 0; shift < 26; shift++) {
        printf("Shift %2d: ", shift);
        decrypt(cipher, shift);
        printf("\n");
    }

    return 0;
}
```

OUTPUT:

```
Enter ciphertext: QSCVHUIJM
Possible plaintexts:
Shift  0: QSCVHUIJM
Shift  1: PRBUGTHIL
Shift  2: OQATFSGHK
Shift  3: NPZSERFGJ
Shift  4: MOYRDQEFI
Shift  5: LNXQCPDEH
Shift  6: KMWPBOCDG
Shift  7: JLVOANBCF
Shift  8: IKUNZMABE
Shift  9: HJTMYLZAD
Shift 10: GISLXKYZC
Shift 11: FHRKWJXYB
Shift 12: EGQJVIWXA
Shift 13: DFPIUHVWZ
```


Q.40:

PROGRAM:

```
    if (isalpha((unsigned char)text[i]))
        freq[toupper((unsigned char)text[i]) - 'A']++;
}

for (int i = 0; i < 26; i++)
    order[i] = i;

/* Sort letters according to frequency */
for (int i = 0; i < 25; i++) {
    for (int j = i + 1; j < 26; j++) {
        if (freq[order[j]] > freq[order[i]]) {
            int t = order[i];
            order[i] = order[j];
            order[j] = t;
        }
    }
}

printf("\nFrequency-based mapping:\n");

for (int i = 0; i < 26; i++) {
    if (freq[order[i]] > 0)
        printf("%c -> %c\n",
            'A' + order[i],
            english[i]);
}

printf("\nThis mapping is a frequency-analysis candidate,\n");
printf("not a guaranteed decryption.\n");

return 0;
}
```

OUTPUT:

```
Enter ciphertext:
QWERTYU

Frequency-based mapping:
E -> E
Q -> T
R -> A
T -> O
U -> I
W -> N
Y -> S

This mapping is a frequency-analysis candidate,
not a guaranteed decryption.

-----
Process exited after 4.946 seconds with return value 0
Press any key to continue . . . |
```